

ARRAYS

It is a data structure which can be defined as finite ordered set of homogeneous elements (fixed no.)
(sequence) (same data type)

- contiguous memory location
- It is the simplest data structure where each element can be accessed directly by its index number.

DECLARATION OF ARRAYS

- Data type - Char, int, float etc
- Name - to identify arrays
- Size - Max no. of values array can hold
- Syntax - type name [size]
- Example -

A	40	50	55	35	42
---	----	----	----	----	----

1	2	3	4	5
---	---	---	---	---

(0 1 2 3 4) index nos

Arrays can be Declared 3 ways

- By specifying size
- int A[10]
- By initializing elements
- int A[] = {10, 5, 4, 3}
- By specifying size and initializing elements
- int A[5] = {10, 20, 30, 40, 50}

INITIALIZATION OF ARRAYS

- i) Initialization of all specified location
 - `int A[5] = { 10 20 30 40 50 }`
 - all 5 memory locations reserved by compiler & all locations initialized
- ii) Partial Array Initialization
 - `int A[5] = { 10 20 }`
 - Compiler allocates 5 memory locations & initializes first two as 10 20 & rest 0
- iii) Initialization without size
 - `char B[] = { 'C', 'O', 'M', 'P' }`
 - Array size is not specified yet it will be set to the total no. of elements initialized. i.e., 4
- iv) Array initialization with String
 - `char B[] = "computer"`
 - Even though the string contains 8 characters but in string it always ends with a null character ∴ size is 9 bytes

ACCESSING ELEMENTS IN AN ARRAY

- First element : eg: `marks[0]`

- We use loop i.e;

```
eg: int i, marks[10]
for (i=0; i<10; i++)
{
```

```
    marks[i] = -1;
```

```
}
```


CALCULATING ADDRESS OF ARRAYS

$$A[K] = B(A) + W(K - LB)$$

A = Array ; K = Index to be calculated

B = Base Address ; W = Size of data type

LB = Lower Bound

Q. int marks[] = {99, 67, 78, 56, 88, 90, 34, 85}

Calculate address of marks[4], BA = 1000

Ans:-

$$A[K] = 4$$

$$B(A) = 1000$$

$$W = 2$$

$$LB = 0$$

$$\therefore \text{marks}[4] = 1000 + 2(4 - 0)$$

$$= 1000 + 8$$

$$= \underline{1008}$$

CALCULATING LENGTH OF ARRAY

$$\text{Length} = UB - LB + 1$$

UB = Index of last element (Upper Bound)

LB = Index of first element (Lower Bound)

Q. Age[5], Age[0]=2, Age[1]=5, Age[2]=3
Age[3]=1, Age[4]=7

Ans.

2	5	3	1	7
0	1	2	3	4

$$\text{length} = UB - LB + 1$$

$$= 4 - 0 + 1$$

$$= 5$$

Questions

1. Let RollNo[3] contains:

RollNo[0] = 101

RollNo[1] = 102

RollNo[2] = 103

- Array layout & Length

Ans i) Array Layout

Value = | 101 | 102 | 103 |

index = 0 1 2

ii) Length

$$= UB - LB + 1$$

$$= 2 - 0 + 1 = 3$$

2. Array: Balance [-2...2] = {100, 200, 300, 400, 500}

Find: i) Draw memory layout from -2 to 2

ii) Calculate array length using bounds

Ans: i) Memory layout

Value = | 100 | 200 | 300 | 400 | 500 |

Index = -2 -1 0 1 2

ii) Length

$$= UB - LB + 1$$

$$= 2 - (-2) + 1 = 5$$

3. Square[i] = $i * i$ for i from 0 to 4

Find = i) write array content

ii) Memory layout iii) length

Ans: i) Square[0] = $0 * 0 = 0$

" [1] = $1 * 1 = 1$

" [2] = $2 * 2 = 4$

" [3] = $3 * 3 = 9$

" [4] = $4 * 4 = 16$

ii) Memory layout

Value = | 0 | 1 | 4 | 9 | 16

Index = 0 1 2 3 4

iii) Length

$$= UB - LB + 1$$

$$= 4 - 0 + 1 = 5$$

STORING VALUES IN ARRAYS

Initialize during
declaration

Input Values
from keyboard

Assign Values to
Individual element

i) Initialize elements during declaration

- When initialized during declaration, a value for every element in array needs to be provided.
- type array-name [size] = { List of values }
- eg: `int marks [4] = { 21, 25, 30, 16 }`
- Array provide $O(1)$ random access lookup time
- Pointer points to right locations of memory
- offset shows how far to look in said memory

array-name [index]

1 | 2 | 3 | 4 | 5
pointed offset

ii) INPUT Values from Keyboard

- While / do while loop is executed to input the value of each elements in array

iii) Assigning values to individual elements

- Using Assignment operators
- eg: `marks [3] = 100;`
(Assigned to 4th element)

TYPES OF ARRAYS

On Basis of
memory Allocation

On the Basis
of Dimensions

→ On Basis of memory Allocation

i) Fixed size arrays

- Cannot update or alter the size of an array.
- Only a fixed size of memory will be allocated for storage - (size mentioned in [])

ii) Dynamic Size Arrays

- The size of an array changes as per user requirement during execution of code
- Can add or remove the elements as per need
- Memory is dynamically allocated and de-allocated

→ On Basis of Dimensions

i) One-dimensional Array (1-D array):

- Imagine it as a row, where elements are stored one after another

Index →	0	1	2	3	4
Elements →	5	4	3	2	1

ii) Multi-Dimensional Array :

- Array with more than one dimension
- To store complex data in form of tables, etc

iii) 2-D Array:

- A array consisting a row & column like a matrix

a_{11} a_{12} a_{13}

a_{21} a_{22} a_{23}

a_{31} a_{32} a_{33}

OPERATIONS ON Arrays

- Traversing : Print all elements one by one
- Inserting an element : Add element at given Index
- Searching an element : Search using index or Value
- Deleting an element : Delete element at given Index
- Merging two Arrays
- Sorting Arrays in Ascending or descending order

ALGORITHM

1) Traversing:

Step 1 : [Initialization] SET $I = \text{lower_bound}$

Step 2 : Repeat step 3 to 4 while $I \leq \text{upper_bound}$

Step 3 : Apply Process to $A[I]$

Step 4 : SET $I = I + 1$

[END OF LOOP]

Step 5 : EXIT

2) Insertion

Step 1 : SET $\text{Upper_Bound} = \text{Upper_bound} + 1$

Step 2 : SET $A[\text{Upper_bound}] = \text{Val}$

Step 3 : EXIT

3) Insertion At Position

Step 1 : [Initialization] SET $I = N$

Step 2 : Repeat step 3 and 4 while $I \geq \text{Pos}$

Step 3 : SET $A[I+1] = A[I]$

Step 4 : SET $I = I - 1$

[END OF LOOP]

Step 5 : SET $N = N + 1$

Step 6 : SET $A[\text{Pos}] = \text{VAL}$

Step 7 : EXIT

4) Deletion At Position

Step 1: [Initialization] SET $I = \text{Pos}$

Step 2: Repeat step 3 to 4 while $I \leq N-1$

Step 3: SET $A[I] = A[I+1]$

Step 4: SET $I = I + 1$

[END OF LOOP]

Step 5: SET $N = N - 1$

Step 6: EXIT

5) Read and Display n number array

Step 1: Initialize & take input 'n'

Step 2: SET $I = 0$

Step 3: Repeat step 4 & 5 while $I < n$

Step 4: Read $A[I]$

Step 5: SET $I = I + 1$

[END OF LOOP]

Step 6: SET $I = 0$

Step 7: Repeat step 8 & 9 while $I < n$

Step 8: Display $A[I]$

Step 9: SET $I = I + 1$

[END OF LOOP]

Step 10: EXIT

6) To Find Second Largest

Step 1: [Initialization] SET $Z = 0$, LARGE, S-LARGE, n

Step 2: Repeat steps 3 & 4 while $Z < n$

Step 3: Read $A[Z]$

Step 4: SET $Z = Z + 1$

[END OF LOOP]

Step 5: SET $Z = 0$

Step 6: Repeat step 7 to 9 while $Z < n$

Step 7: IF $A[Z] > \text{LARGE}$ then

SET $\text{S_LARGE} = \text{LARGE}$

SET $\text{LARGE} = A[Z]$

Step 8: ELSE IF $A[Z] > \text{S_LARGE}$

AND $A[Z] \neq \text{LARGE}$ then

SET $\text{S_LARGE} = A[Z]$

Step 9: SET $Z = Z + 1$

[END OF LOOP]

Step 10: DISPLAY S_LARGE

Step 11: EXIT

MULTI DIMENSIONAL ARRAYS

- referenced by more than 1 subscript
- Collection of $M \times N$ specified by j, k Subscript
- Denoted by $A[j][k]$
- $m = \text{rows}$, $N = \text{columns}$

To Print elements in 2D Array

Step 1: [Initialization] SET $I = 0$, SET $J = 0$

Step 2: Repeat step 3 to 6 while $I < \text{rows}$

Step 3: Print new lines

Step 4: SET $J = 0$

Step 5: Repeat while $J < \text{col}$

Print $A[I][J]$

SET $J = J + 1$

Step 6: SET $I = I + 1$

[END OF LOOP]

Step 7: EXIT

CALCULATING ADDRESS IN 2-D ARRAY

$$A[i][j] = B + ((i * n) + j) * \text{Size}$$

↓
(Row major)

$$A[i][j] = B + ((j * m) + i) * \text{size}$$

↓
(column major)

B = Base address m = rows n = columns

i = row index

j = column index $\text{size} = \text{datatype}$

$B = 100$ $\text{int } A[2][3]$

$A[1][1]$

$$\begin{aligned} \therefore A[i][j] &= 100 ((1 * 3) + 1) * 4 \\ &= 100 + (3 + 1) * 4 \\ &= 116 \end{aligned}$$

STACK

- Data structure which stores its element in an ordered manner
- Stack is a linear data structure
- The elements in the stack are added & remove only from one end which is called TOP
- Stack is called a LIFO (Last in First Out)

VARIOUS OPERATIONS ON STACK

- PUSH
- POP
- PEEK
- isEmpty()
- isFull()
- Can be implemented using Arrays or using linked list

ALGORITHMS

1) PUSH

Step 1 : IF $TOP = MAX - 1$

PRINT "OVERFLOW"

GO TO Step 4

[END OF IF]

Step 2 : SET $TOP = TOP + 1$

Step 3 : SET $STACK[TOP] = VALUE$

Step 4 : END

2) POP

Step 1 : IF $TOP = NULL$
PRINT "UNDERFLOW"

Goto step 4

[END OF IF]

Step 2 : SET $VAL = STACK[TOP]$

Step 3 : SET $TOP = TOP - 1$

Step 4 : END

3) PEEK

Step 1 : IF $TOP = NULL$

~~Step 2~~ PRINT "EMPTY"

Goto Step 3

Step 2 : RETURN $STACK[TOP]$

Step 3 : EXIT

4) $ISEMPTY()$ & $ISFULL()$

i) IF $TOP == -1$, stack is EMPTY

return TRUE else FALSE

ii) IF $TOP = N - 1$, stack is FULL

return TRUE else FALSE

COMPLEXITY ANALYSIS

Time Complexity :

• Push : $O(1)$

• Pop : $O(1)$

• Peek : $O(1)$

• Is-empty : $O(1)$

• Is-full : $O(1)$

Auxiliary Space : $O(n)$, where n is the number of items in stack

REVERSING A LIST

Step 1 : START

Step 2 : Initialize an empty stack STACK[]

SET TOP = -1

Step 3 : Push elements of ARR[] into STACK[]

FOR $i = 0$ to $N-1$ DO

TOP = TOP + 1

STACK[TOP] = ARR[i]

END FOR

Step 4 : Pop all elements from STACK[] back to ARR[]

FOR $i = 0$ to $N-1$ DO

ARR[i] = STACK[TOP]

TOP = TOP - 1

END FOR

Step 5 : END

- Create an empty stack
- Push all elements of array to stack
- Pop all elements one by one
- Push all the popped elements back

i) Function Call Management

- When function is called, its execution context (local variables, return address) is pushed onto call stack. When function returns, context is popped off. This is recursion internally.

ii) Expression Evaluation

- Used to evaluate arithmetic expression i.e. prefix & postfix. They help in converting infix $\rightarrow$ postfix & evaluating them.

iii) Syntax parsing

- Compilers use stacks to parse syntax & check for balanced paranthesis, brackets.

iv) Undo Mechanism

- Text editors use stack for undo application. Each actions is pushed onto stack, & undo pops the last action.

v) Backtracking Algorithms

- Stacks are used in algorithms that include maze solving, puzzles research graph etc.

vi) Browser History

- Web browser use stacks to manage history. Most recently visited pages is on top & going back pops the current page.

ADVANTAGES

- Simple & efficient: Easy to implement & use
- Memory Management: Helps in managing memory during recursive function calls
- Controlled Access: Access only to top element, ensuring data integrity
- Versatile: Useful in wide range of application from system level programming to user interface

DISADVANTAGES

- Limited access: Access only to top element
- Overflow & Underflow: Fix sized stacks can overflow & popping from empty stack causes underflow
- Not ideal for random Access: Unlike arrays, stacks do not support random access to elements

INFIX, POSTFIX & PREFIX NOTATION

- Infix: Operator placed btw operands
- Postfix: Operator placed after operands
 - Order of evaluation: Left - right
- Prefix: Operator is placed before operands

PRECEDENCE

- $()$
- $^$; Associativity: Right - Left
- $/, *$; Associativity: Left - Right
- $+, -$; Associativity: Left - Right

Q. Infix to Prefix

i) $A * B + C / D$

$$= + * AB + C / D$$
$$= + * AB + / CD$$
$$= + * AB / CD$$

ii) $A + B * C$

$$= A + * BC$$
$$= + A * BC$$

iii) $A + B - C$

$$= + AB - C = - + ABC$$

iv) $(3 + 4) * (8 - 9)$

$$= + 3 4 * - 8 9$$
$$= * + 3 4 - 8 9$$

v) $x / y ^ z + A$

$$= x / ^ y z + A$$
$$= / x ^ y z + A = + / x ^ y z + A$$

INFIX TO ~~AT~~ PREFIX

$$K + L - M * N + (O^{\wedge}P)^{*}W / U / V^{*}T + Q$$

Reverse :

$$Q + T^{*}V / U / W^{*}(P^{\wedge}O) + N^{*}M - L + K$$

Input	Stack	Prefix
Q		Q
+	+	Q
T	+	QT
*	+*	QT
V	+*	QTV
/	+*/	QTV
U	+*/	QTVU
/	+**	QTVU
W	+**	QTVUW
*	+**	QTVUW
(	+**	QTVUW
P	+**	QTVUWP
^	+**	QTVUWP
O	+**	QTVUWPO
)	+**	QTVUWPO^
+	++	QTVUWPO^* **
N	++	QTVUWPO^* ** N
*	++*	QTVUWPO^* ** N
M	++*	QTVUWPO^* ** NM
-	++-	QTVUWPO^* ** NM*
L	++-	QTVUWPO^* ** NML
+	++-+	QTVUWPO^* ** NML
K	++-+	QTVUWPO^* ** NMLK
		QTVUWPO^* ** NMLK + - ++
Reversed -		++-+KL*MN* ** ^ OPWUV T Q

INFIX To POSTFIX

$$A - (B/C + (D \% E * F) / G) * H$$

Input	Stack	Postfix
A		A
-	-	A
(	- (	A
B	- (	AB
/	- (/	AB
C	- (/	ABC
+	- (+	ABC /
(	- (+ (	ABC /
D	- (+ (	ABC / D
%	- (+ (%	ABC / D
E	- (+ (%	ABC / DE
*	- (+ (* %	ABC / DE %
F	- (+ (* %	ABC / DE % F
)	- (+ (* %	ABC / DE % F *
/	- (+ / %	ABC / DE % F *
G	- (+ / %	ABC / DE % F * G
)	- (+ / %	ABC / DE % F * G / +
*	- (+ / %	ABC / DE % F * G / +
H	- (+ / %	ABC / DE % F * G / + H
Answer :-		ABC / DE % F * G / + H * -

Infix to Postfix (without stack)

$$\begin{aligned}
 & (a+b-c) * d - (e+f) \\
 &= (ab+-c) * d - (ef+) \\
 &= (ab+c-)* d - (ef+) \\
 &= ab+c-d* - ef+ \\
 &= ab+c-d* ef+-
 \end{aligned}$$

ALGORITHM

→ Infix to Prefix

Step 1: Scan each character in infix
repeat steps 2 to 8

Step 2: Push operator into operator oprand into operator stack, all left right paranthesis encounter

Step 3: Pop oprand 2 from oprand stack

Step 4: Pop operand 1 from operand stack

Step 5: Pop operand from operand stack

Step 6: Concatenate operator & operand 1

Step 7: Concatenate operator 2 & result

Step 8: Push result into operand stack

Step 9: END

→ Infix to Postfix

Step 1: Add) end of infix expression

Step 2: Push (onto stack

Step 3: Repeat until each infix notation scanned

- IF a (is encountered push on stack

- IF an operand (digit / character), add to postfix exp

- IF a) encountered then,

i) - repeatedly pop from stack and push to postfix until) encountered

ii) Discard (

- IF operator encountered

- i) Repeatedly Pop from stack & Add it to postfix which has same or higher precedence
- ii) Push operator to stack

Step 4: Repeatedly pop from stack & add to postfix until stack is empty

Step 5: Exit

BALANCE PARANTHESIS CHECKER

ALGO:-

Step 1: Initialize an empty stack

Step 2: Scan expression left to right

Step 3: For each character

a) if opening bracket, push into stack

b) if closing bracket:

i) check if stack non empty

ii) Pop top element & check if matches the type of closing bracket

iii) If not match or empty stack, then expression unbalanced

Step 4: After scanning if stack empty then balance else not balanced

Q. { [()] }

Symbol	Stack
{	{
[	{ [
(	{ [(
)	{ [
]	{
}	empty

= Balanced

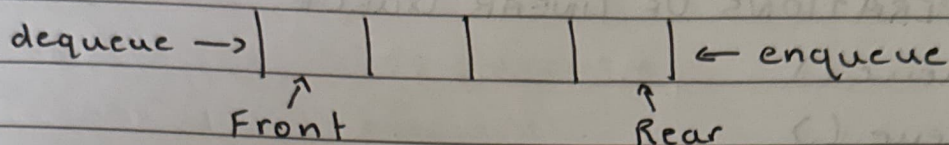
Q. { [(< >] }

Symbol	Stack
{	{
[	{ [
(	{ [(
!	{ [(!
)	{ [()
}	{ [

= not balanced

QUEUE

- It is an ordered list which enables insert operations to be performed at one end called Rear and delete to be performed by other end called Front
- FIFO (First in First Out)



Applications Of QUEUE

- Used as waiting list for a single shared resource like printer, disk, CPU
- Used in asynchronous transfer of data eg. pipes
- Used as buffers in most of application like MP3, CD
- In media player to add or remove songs from playlist
- Used in operating systems for handling interrupts

TYPES OF QUEUE

- Simple Queue (Linear)
- Circular Queue
- Priority Queue
- Double-Ended Queue

SIMPLE QUEUE (LINEAR)

Insertion takes place from one end while the deletion occurs from another end. Insertion from rear & deletion from Front. FIFO

Characteristics OF LINEAR QUEUE

1. FIFO order
2. Single Direction : Insertion rear ; deletion Front
3. Fixed Size : Has max capacity
4. Sequential Storage : Stored in linear sequence

BASIC OPERATIONS OF LINEAR QUEUE

1. Enqueue ()
2. Dequeue ()
3. front ()
4. isEmpty ()
5. isFull ()

DISADVANTAGE

- Insertion only done by from rear end
- If First three elements deleted we cannot insert more elements even if space available
- Fixed Size, wasted SPACE, No reuse of space

ALGORITHM

1. Initialize Queue

- Step 1: Define MAX_SIZE as max capacity of queue
Step 2: Create an array QUEUE of size MAX_SIZE
Step 3: Set FRONT = -1 & REAR = -1

2. Enqueue

- Step 1: IF REAR = MAX_SIZE - 1 then
 DISPLAY "OVER FLOW"
Step 2: IF REAR = -1, then
 front = REAR = 0

ELSE

 REAR = REAR + 1

Step 3: QUEUE[REAR] = val

Step 4: EXIT

3. Dequeue

Step 1: IF $FRONT == -1$, then
 DISPLAY "UNDERFLOW"

Step 2: $ELEMENT = QUEUE[FRONT]$

Step 3: IF $FRONT = REAR$ Then

$FRONT = REAR = -1$

ELSE

$FRONT = FRONT + 1$

Step 4: Return Element

Step 5: END

4. FRONT

Step 1: IF $FRONT == -1$ then

DISPLAY "UNDERFLOW"

Step 2: $ELEMENT = QUEUE[FRONT]$

Step 3: Return ELEMENT

Step 4: END

5. is Empty

Step 1: IF $FRONT == -1$

Return TRUE

Step 2: Else

Return False

Step 3: END

6. is FULL

Step 1: IF $REAR == MAX_Size - 1$ then

Return TRUE

Step 2: Else

Return False

Step 3: END

ADVANTAGES OF LINEAR QUEUE

- Simple Structure: Easy to use
- Orderly Processing: Tasks are handled in order they arrive
- Efficient for Sequential Task: Ideal for scenario where order matters

CIRCULAR QUEUE

- The Last element of the queue is connected to the first element.
- All nodes are represented as circular

CHARACTERISTICS OF CIRCULAR

1. FIFO order: Elements processed in order they arrive
2. Circular arrangement: Last position connected to first
3. Efficient memory use: No waste space
4. Fixed size:
5. Two pointers

BASIC OPERATIONS

1. En Queue
2. De Queue
3. Front
4. Is Empty
5. Is Full

ALGORITHMS

1. Initialize Queue()

Step 1: Define Max_Size

Step 2: Create an ARRAY array QUEUE of size MAX-SIZE

Step 3: Set FRONT = -1 & REAR = -1

2. En Queue ()

Step 1: IF $(\text{FRONT} = 0 \ \& \ \text{REAR} = \text{MAX_SIZE} - 1)$ OR $(\text{FRONT} = \text{REAR} + 1)$

Display "Overflow"

Goto Step 4

Step 2: IF $\text{REAR} = -1$ Then

$\text{FRONT} = \text{REAR} = 0$

ELSE IF $\text{REAR} = \text{MAX_SIZE} - 1$, Then

$\text{REAR} = 0$

ELSE

$\text{REAR} = \text{REAR} + 1$

Step 3: $\text{Queue}[\text{REAR}] = \text{Element}$

Step 4: END

3. De Queue ()

Step 1: IF $\text{FRONT} = -1$ Then

Display "Underflow" & Goto step 5

Step 2: $\text{Element} \leftarrow \text{QUEUE}[\text{FRONT}]$

Step 3: IF $\text{FRONT} = \text{REAR}$ Then

$\text{FRONT} = \text{REAR} = -1$

ELSE IF $\text{FRONT} = \text{MAX_SIZE} - 1$, Then

$\text{FRONT} = 0$

ELSE

$\text{FRONT} = \text{FRONT} + 1$

Step 4: Return Element

Step 5: END

4. FRONT ()

Step 1: IF $\text{FRONT} == -1$ Then

Display "Underflow" & Go To step 4

Step 2: $\text{Element} = \text{Queue}[\text{FRONT}]$

Step 3: Return Element

Step 4: END

5. Is Empty ()

Step 1: IF FRONT == -1 Then

Return TRUE

ELSE

Return FALSE

Step 2: END

6. Is Full ()

Step 1: IF (FRONT == 0 & REAR == MAX_SIZE - 1) or (FRONT == REAR - 1)

Return TRUE

ELSE

Return FALSE

Step 2: END

ADVANTAGES

- Efficient Memory Utilization: No waste space
- Fixed Size Control: Ideal for buffer management
- Fast Operations: Constant time insertion & deletion
- Predictable Behaviour: Useful in real time systems

DISADVANTAGES

- Complex pointer Management: Requires careful handling
- Fixed Capacity: Cannot grow dynamically unless with linked list
- Overflow Risk: Must check for full condition before enqueue

APPLICATIONS

1. CPU scheduling
2. Buffer Management
3. Traffic Systems
4. Embedded Systems
5. Multitasking Systems

CIRCULAR QUEUE VS LINEAR QUEUE

Feature	Linear Queue	Circular Queue
Memory Utilization	Inefficient	Efficient
Wrap-around Support	No	Yes
Overflow Risk	High	Low
Implementation	Simple	Slightly Complex
Use case	Basic scheduling	Real-time Systems

PRIORITY QUEUE

- Queue in which each element is associated with a priority (abstract data type)
- Priority of an element will be used to determine the order in which these elements are processed
- Element with higher priority is processed before an element with lower
- Two with same priority are processed on a first come first served basis

CHARACTERISTICS OF PRIORITY QUEUE

1. Priority - Based Processing : Elements are dequeued based on priority, not arrival time
2. Dynamic Ordering : It reorders itself as elements are added or removed
3. Multiple Priority : Each element has a priority value, which determines its position
4. Flexible implementation : Can be implemented using arrays, linked list, heaps or binary trees

BASIC OPERATIONS OF PRIORITY QUEUE

1. En Queue
2. De Queue
3. Peek
4. is Empty
5. Change Priority: Updates the priority of an existing element

TYPES OF PRIORITY QUEUE

1. MAX priority Queue
In this the element with highest priority is served first
2. MIN priority Queue
In this the element with lowest priority value is served

IMPLEMENTATION OF PRIORITY QUEUE

1. Using Arrays or linked lists
 - Unsorted Array/List: Insertion is fast ($O(1)$), but deletion requires scanning the entire structure ($O(n)$)
 - Sorted Array/List: Deletion is fast ($O(1)$), but insertion requires finding the correct position ($O(n)$)
2. Using heaps

Most efficient and common implementation is using binary heap, which is a complete binary tree that maintains the heap property:

 - Max Heap: Parent node is greater than or equal to its children
 - Min Heap: Parent node is less than or equal to its children

OPERATIONS In HEAP-BASED PRIORITY QUEUE

- Insert : $O(\log n)$
- Delete Max/Min : $O(\log n)$
- Peek : $O(1)$

APPLICATIONS OF PRIORITY QUEUE

1. CPU scheduling
2. Dijkstra's Algorithm
3. Bandwidth management
4. Emergency Systems
5. Event - Driven Simulations
6. Data Compression

ADVANTAGE

- Efficient Task management: Ensures imp task handled first
- Flexible Design: Can be tailored to specific needs
- Optimized Performance: Heap-based implementations offer logarithmic time complexity
- Widely Applicable: Useful in both system-level and application-level programming

DIS-ADVANTAGE

- Complex Implementation: More complex than linear queues
- Overhead in Maintenance: Requires reordering after ^{insertion or} deletion
- Limited Direct Access: Can not access arbitrary elements easily